

Lab - 2:

Experiment No: 2

Dated:

AIM: Create a Blockchain using Python

Lab Objectives: To implement public and private Blockchain.

Lab Outcomes (LO): Demonstrate the concept of Blockchain in real-world Applications (LO4)

Task to be performed :

1. Make a copy of this [Google Colab Notebook](#)
2. Try to solve the errors in given Program
3. After successful execution of the Program in Colab Notebook.
 - **Add a method, create_Transactions**
 - Mine the block only when the transaction list is not null.
 - Remove the transactions from the list of transactions before mining.
 - **Modify the method, proof_of_work() to search for the golden nonce**
 - Cryptographic Puzzle is to have “000” leading zeros in the Block Hash
4. Download the code - [blockchain.py](#)
5. **Update the code to incorporate the changes in step 3 to the code in step 4.**
6. Follow the steps in [Manual](#) to demonstrate the working of Blockchain using Flask and Postman.

Tools & Libraries used :

- [Flask](#) : pip install Flask
- Download **Postman** from <https://www.postman.com/>
- Python Libraries : **datetime, jsonify, hashlib**

Instructions : (Prepare for viva for the following topics)

1. What is a Blockchain?
2. Process of Mining
3. How to check the validity of blocks in a Blockchain

Outcome :

1. Understood the concept of mining in Blockchain.
2. Implemented programs
 - Mine a Block
 - Display the chain in the Blockchain
 - Check whether the chain is valid
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**
(As per the instructions, submit a hard copy of the same).

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

1. Importing Required Libraries

- **datetime** → to timestamp each block
- **hashlib** → to generate SHA-256 hashes
- **json** → to encode blocks as JSON for hashing
- **Flask** → to run the blockchain API server

2. Blockchain Class – Core Logic

This contains everything needed to:

- create blocks
 - store the chain
 - implement Proof of Work
 - validate chain integrity
-

3. Flask Web API

Provides endpoints:

- `/mine_block` → mine a new block
- `/get_chain` → fetch full blockchain
- `/is_valid` → verify blockchain

Program And Output

🔗 Blockchain_Lab_2.ipynb

```
import datetime
```

```
import hashlib
```

```
import json
```

```
class Blockchain:
```

```
    def __init__(self):
```

```
        self.chain = []
```

```

        self.create_block(proof = 1, previous_hash = '0')

def create_block(self, proof, previous_hash):
    block = {'index': len(self.chain) + 1,
             'timestamp': str(datetime.datetime.now()),
             'proof': proof,
             'previous_hash': previous_hash}

    self.chain.append(block)

    return block

def get_previous_block(self):
    return self.chain[-1]

def proof_of_work(self, previous_proof):
    new_proof = 1
    check_proof = False

    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()

        if hash_operation[:4] == '0000':
            check_proof = True

        else:

```

```

        new_proof += 1

    return new_proof

def hash(self, block):

    encoded_block = json.dumps(block, sort_keys =
True).encode()

    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):

    previous_block = chain[0]

    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        if block['previous_hash'] !=
self.hash(previous_block):

            return False

        previous_proof = previous_block['proof']

        proof = block['proof']

        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()

        if hash_operation[:4] != '0000':

            return False

        previous_block = block

```

```

        block_index += 1

    return True

# Creating a Blockchain

blockchain = Blockchain()

# Mining a new block

def mine_block():

    previous_block = blockchain.get_previous_block()

    previous_proof = previous_block['proof']

    proof = blockchain.proof_of_work(previous_proof)

    previous_hash = blockchain.hash(previous_block)

    block = blockchain.create_block(proof, previous_hash)

    response = {'message': 'Congratulations, you just mined a
block!',

                'index': block['index'],

                'timestamp': block['timestamp'],

                'proof': block['proof'],

                'previous_hash': block['previous_hash']}

    return response

# Getting the full Blockchain

def get_chain():

    response = {'chain': blockchain.chain,

                'length': len(blockchain.chain)}

```

```

        return response

    # Checking if the Blockchain is valid

def is_valid():

    is_valid = blockchain.is_chain_valid(blockchain.chain)

    if is_valid:

        response = {'message': 'All good. The Blockchain is
valid.'}

    else:

        response = {'message': 'Houston, we have a problem. The
Blockchain is not valid.'}

    return response


print("Functions Menu:")

print("=====")

print("1. Mine a block")

print("2. Display the chain")

print("3. Check the validity of the chain")

choice = int(input("Enter your choice :"))

if choice == 1:

    print(mine_block())

elif choice == 2:

    print(get_chain())

elif choice == 3:

```

```
print(is_valid())

else :

    print("Invalid choice")
```

Functions Menu:

=====

1. Mine a block
2. Display the chain
3. Check the validity of the chain

Enter your choice :2

```
{'chain': [{ 'index': 1, 'timestamp': '2026-08-27 04:14:15.832321',
'proof': 1, 'previous_hash': '0'}, { 'index': 2, 'timestamp': '2026-08-27
04:17:35.746900', 'proof': 533, 'previous_hash':
'5339e3cb12bf6a62478eb32d4f367d9a3dce375dc62ca3d99e5356542819d068'}]},
'length': 2}
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.